FORMATION FLUTTER / APRÈS LES BASES DART

Premiers écrans, premières interactions

Support élève

Mardi 20 octobre 2026

Écran → layout → liste → favoris → fiche → formulaire → attente et erreur → adaptation et recette

21 notions • 63 questions progressives • une application • 11 jalons cumulatifs

Le projet final montre un catalogue fictif. Les demandes sont simulées et les données restent en mémoire. Les fichiers sont inclus dans le kit séparé et en pièce jointe de ce PDF.

Essayer les exercices avant de lire les corrigés de quiz regroupés en fin de support. Garder son travail avant toute reprise depuis un checkpoint.

Sommaire et navigation

Sommaire et navigation 2

Parcours du cours 5

Démarrer le projet et choisir une étape 5

Les onze jalons du même projet . . . 5

1. Découvrir Flutter 6

F01 - De Dart à Flutter : lancer un premier écran . . . 6

F02 - Enregistrer, hot reload et hot restart . . . 6

F03 - Widget, arbre, build et BuildContext . . . 7

F04 - StatelessWidget et paramètres nommés . . . 8

F05 - MaterialApp, Scaffold, thème et premiers widgets . . . 9

2. Composer et contraindre 11

F06 - Row, Column et composition d’une carte . . . 11

F07 - Contraintes, Expanded et débordements . . . 11

3. Données, état et callbacks 13

F08 - Des objets Dart vers une liste défilante . . . 13

F09 - StatefulWidget, State et setState . . . 13

F10 - Reconstruire sans perdre l’état : final et const . . . 14

F11 - Extraire CarteArtisan et transmettre des callbacks . . . 15

4. Naviguer, saisir et attendre 17

F12 - Naviguer vers une fiche et récupérer un résultat . . . 17

F13 - Form et TextFormField : valider avant d’agir . . . 17

F14 - Contrôleur et cycle de vie : créer une fois, libérer . . . 18

F15 - Afficher attente, succès et erreur avec Future . . . 19

F16 - mounted et retour arrière pendant une attente . . . 20

5. Adapter, vérifier et intégrer 21

F17 - Adapter le layout à la largeur disponible . . . 21

F18 - Accessibilité : nommer les actions et agrandir le texte . . . 21

F19 - Lire une erreur et vérifier un comportement . . . 22

F20 - Atelier autonome : relier recherche et favoris . . . 23

F21 - Recette finale et transmission vers mercredi . . . 24

Récapitulatif du cours suivi 25

Lexique à garder sous la main . . . 25

Checklist de recette et reprise personnelle . . . 26

Corrigés des quiz - après sa tentative 27

F01 - De Dart à Flutter : lancer un premier écran . . . 27

F02 - Enregistrer, hot reload et hot restart . . . 27

F03 - Widget, arbre, build et BuildContext . . . 27

F04 - StatelessWidget et paramètres nommés . . . 27

F05 - MaterialApp, Scaffold, thème et premiers widgets . . . 27

F06 - Row, Column et composition d’une carte . . . 28

F07 - Contraintes, Expanded et débordements . . . 28

F08 - Des objets Dart vers une liste défilante . . . 28

F09 - StatefulWidget, State et setState . . . 28

F10 - Reconstruire sans perdre l’état : final et const . . . 28

F11 - Extraire CarteArtisan et transmettre des callbacks . . . 29

F12 - Naviguer vers une fiche et récupérer un résultat . . . 29

F13 - Form et TextFormField : valider avant d’agir . . . 29

F14 - Contrôleur et cycle de vie : créer une fois, libérer . . . 29

F15 - Afficher attente, succès et erreur avec Future . . . 29

F16 - mounted et retour arrière pendant une attente . . . 30

F17 - Adapter le layout à la largeur disponible . . . 30

F18 - Accessibilité : nommer les actions et agrandir le texte . . . 30

F19 - Lire une erreur et vérifier un comportement . . . 30

F20 - Atelier autonome : relier recherche et favoris . . . 30

F21 - Recette finale et transmission vers mercredi . . . 31

Fichiers organisés et commandes utiles **32**

lib . . . 32

test . . . 32

Repères officiels et suite du parcours **33**

Utiliser les titres du sommaire ou les signets du lecteur. Le code est sélectionnable ; les fichiers du projet conservent exactement les imports et l’indentation.

Parcours du cours

Construire un premier écran, comprendre son arbre, composer une carte, présenter des données, faire évoluer les favoris, ouvrir une fiche, valider une saisie et gérer une réponse asynchrone. Terminer par l'adaptation du layout et une recette expliquée.

Démarrer le projet et choisir une étape

Décompresser le kit dans un nouveau dossier et ouvrir helpme_mardi dans l'éditeur. Exécuter les commandes dans ce dossier, celui de pubspec.yaml. Notre app commence avec un écran Bonjour et conserve le même point d'entrée pendant toute la journée.

```
flutter --version
flutter pub get
flutter devices
flutter run -d chrome
```

Le point d'entrée reste lib/main.dart. Modifier ou ajouter les fichiers indiqués DANS notre dossier lib courant. Enregistrer puis utiliser r. Lorsqu'un widget change de StatelessWidget vers StatefulWidget, utiliser R. Un redémarrage remet l'état en mémoire à zéro ; il ne supprime aucun fichier.

A chaque jalon, on garde les fonctionnalités réalisées : la liste garde la carte, les favoris restent quand on extrait le composant, la fiche garde ses informations quand le formulaire arrive, et la recherche s'ajoute aux favoris déjà terminés. On ne relance pas un autre fichier d'étape.

Le guide parcours/PAS_A_PAS.md et les onze fiches de parcours indiquent les modifications à intégrer et les vérifications à conserver. Leurs blocs de référence servent à comparer un fichier après le jalon ; ne pas remplacer tout son travail pour avancer. Les réglages personnels restent dans lib/configuration.dart.

Les onze jalons du même projet

Jalon	Fiches	Ajout à notre application
01_accueil	F01, F02, F03	Notre premier écran
02_parametres	F04	Ajouter un composant sans effacer notre écran
03_presentation	F05, F06, F07	Ajouter une carte à notre page
04_catalogue	F08	Passer d'une carte à une liste
05_favoris	F09, F10	Rendre notre catalogue interactif
06_composants	F11	Extraire la carte, garder les favoris
07_navigation	F12	Ajouter une fiche au catalogue existant
08_formulaire	F13, F14	Enrichir la même fiche avec un formulaire
09_asynchrone	F15, F16	Ajouter attente et erreur au même formulaire
10_responsive	F17, F18, F19	Adapter notre application déjà fonctionnelle
11_recherche	F20, F21	Ajouter la recherche, garder tout le parcours

1. Découvrir Flutter

F01 - De Dart à Flutter : lancer un premier écran

Repère de cours

Dart est le langage appris lundi. Flutter fournit des classes et un moteur pour construire une interface. main reste le point d'entrée ; runApp reçoit le premier widget de l'application. Un print apparaît dans la console, tandis qu'un Text décrit du texte à afficher à l'écran.

Un projet Flutter possède un pubspec.yaml, des sources dans lib et des dossiers de plateformes. Le SDK fournit les widgets Material. Les exemples du jour utilisent Chrome pour le premier lancement et ne demandent ni compte, ni serveur, ni API distante.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
void main() {
  runApp(const HelpMeApp());
}
// Dans lib/configuration.dart :
const messageAccueil = 'Bonjour Flutter !';
```

Manipulation élève

Lancer notre application avec flutter run -d chrome. Personnaliser messageAccueil dans lib/configuration.dart. Ajouter un debugPrint temporaire avant runApp dans main, repérer le terminal puis retirer le log.

Continuité : jalon 01_accueil ; guide parcours/01_accueil.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/main.dart

Preuve attendue : Le titre et le message personnalisés apparaissent. Un Text affiché et un log exécuté ont deux destinations différentes.

Quiz - trois questions de difficulté croissante

1. Repérer - Dart et Flutter ont-ils le même rôle ?

2. Prédire - Où voit-on un Text inséré dans l’arbre et un debugPrint exécuté ?

```
// Dans l'arbre affiché : const Text('Bonjour')
// Dans une action exécutée :
debugPrint('Bonjour');
```

3. Diagnostiquer / transférer - Pourquoi dart run ne remplace-t-il pas flutter run pour cet écran ?

Ma justification : .....
.....

F02 - Enregistrer, hot reload et hot restart

Repère de cours

En debug, hot reload injecte les changements de code et conserve généralement l'état existant. Dans le terminal de flutter run, r déclenche le rechargement. Hot restart, avec R, relance l'application Dart et perd l'état en mémoire. q quitte. Le support de l'éditeur peut aussi proposer ces actions.

Hot reload ne réexécute pas main ni initState. Une modification du code natif ou de certaines initialisations demande un redémarrage adapté. Enregistrer ne suffit pas dans tous les éditeurs : observer le résultat et le message de rechargement.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
flutter run -d chrome
# r : recharger ; R : redémarrer ; q : quitter
```

Manipulation élève

Dans configuration.dart, modifier le message et le titre, enregistrer puis utiliser r. Repérer R et q. Garder le même point d’entrée pour toute la journée.

Continuité : jalon 01_accueil ; guide parcours/01_accueil.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/main.dart

Preuve attendue : Le titre et le message personnalisés apparaissent. Un Text affiché et un log exécuté ont deux destinations différentes.

Quiz - trois questions de difficulté croissante

1. Repérer - Quelle commande déclenche hot reload au terminal ?

2. Prédire - Les favoris en mémoire sont-ils normalement conservés par hot reload ?

3. Diagnostiquer / transférer - Une nouvelle initialisation dans main n’est pas appliquée : quelle action essayer ?

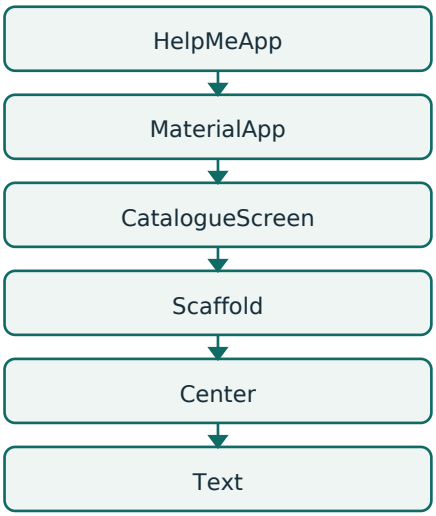
Ma justification : .....
.....

F03 - Widget, arbre, build et BuildContext

Repère de cours

Un widget est une description de configuration de l'interface. Les widgets s'emboîtent en arbre : MaterialApp contient une page, qui contient d'autres widgets. build reçoit un BuildContext et renvoie un Widget. Il peut être rappelé : on y décrit le résultat, sans envoyer une demande réseau à chaque passage.

BuildContext repère une position dans l'arbre. Il permet de retrouver des services situés au-dessus, par exemple un thème ou un navigateur. Un widget n'est pas l'objet qui dessine seul tous les pixels : Flutter entretient aussi des éléments et des objets de rendu. Ce premier cours utilise surtout l'arbre des widgets.



Branche principale du premier écran : HelpMeApp → MaterialApp → CatalogueScreen → Scaffold → Center → Text. Le Padding ajouté pendant la manipulation devient un parent de Text.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
MaterialApp(
  home: Scaffold(
    body: Center(child: Text('Bonjour')),
  ),
)
```

Manipulation élève

Dans notre CatalogueScreen, dessiner MaterialApp → CatalogueScreen → Scaffold → Center → Text. Entourer le Text par un Padding sans perdre le message personnalisé.

Continuité : jalon 01_accueil ; guide parcours/01_accueil.md. Le travail reste dans lib et le point d'entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : Le titre et le message personnalisés apparaissent. Un Text affiché et un log exécuté ont deux destinations différentes.

Quiz - trois questions de difficulté croissante

1. Repérer - Quel type build doit-il renvoyer ?

2. Prédire - Quel est le parent direct de Text dans cet extrait ?

```
Center(child: Text('Bonjour'))
```

3. Diagnostiquer / transférer - Pourquoi ne pas lancer une sauvegarde directement dans build ?

Ma justification : .....
.....

F04 - StatelessWidget et paramètres nommés

Repère de cours

Un StatelessWidget reçoit une configuration et décrit son affichage. Il peut être reconstruit même s'il ne possède pas son propre état mutable. Les paramètres nommés, required, final et le constructeur sont les notions Dart de lundi appliquées à l'interface.

child attend un seul Widget ; children attend une liste de widgets. const peut décrire une configuration constante quand ses arguments le permettent. Cela ne signifie pas que toute l'application serait figée. super.key est fourni pour transmettre une clé au widget parent ; les détails d'identité seront approfondis après les premiers écrans.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
class TitreSection extends StatelessWidget {
  const TitreSection({super.key, required this.titre});
  final String titre;
  @override
  Widget build(BuildContext context) => Text(titre);
}
```

Manipulation élève

Créer TitreSection avec un champ final titre et un constructeur required. Dans notre catalogue, remplacer le Center par une Column contenant deux TitreSection de textes différents et notre message personnel. Ne pas modifier main.

Continuité : jalon 02_parametres ; guide parcours/02_parametres.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/widgets/titre_section.dart

Preuve attendue : Deux titres différents utilisent la même classe. Le titre de l’application et le message personnel sont conservés.

Quiz - trois questions de difficulté croissante

1. Repérer - StatelessWidget veut-il dire « jamais reconstruit » ?

2. Prédire - Quel type attend children ?

3. Diagnostiquer / transférer - Pourquoi cet appel sans titre est-il refusé ?

```
// Constructeur : const TitreSection({super.key, required this.titre});
// À diagnostiquer : const TitreSection();
```

Ma justification : .....
.....

F05 - MaterialApp, Scaffold, thème et premiers widgets

Repère de cours

MaterialApp fournit le thème et l'environnement de navigation Material. Scaffold structure une page avec appBar et body. Text affiche du texte ; Icon utilise une icône intégrée ; Padding ajoute un espace intérieur ; SizedBox peut réserver un espace. Le thème permet de partager des choix visuels.

Une interface commence par une hiérarchie et des noms lisibles, avant l'accumulation de réglages. Theme.of(context) lit un thème ancêtre à partir de la position du contexte. Un contexte situé au-dessus de l'application ne voit pas les services qu'elle est en train de retourner.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
MaterialApp(
  theme: ThemeData(colorSchemeSeed: Colors.teal),
  home: Scaffold(
    appBar: AppBar(title: const Text('HelpMe')),
```

```
    body: const Padding(
      padding: EdgeInsets.all(20),
      child: Text('Trouver un artisan'),
    ),
  ),
)
```

Manipulation élève

Garder les titres et le message. Changer la couleur du thème dans app.dart. Préparer une zone de contenu dans CatalogueScreen avec SafeArea, Padding, Column et SizedBox, puis reprendre la carte au jalon 03.

Continuité : jalon 03_presentation ; guide parcours/03_presentation.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : L’en-tête et le message sont toujours là. La première carte montre Gérard, Plomberie et son tarif fictif. Un nom long reste dans la largeur disponible.

Quiz - trois questions de difficulté croissante

1. Repérer - Quel widget organise appBar et body ?

2. Prédire - Padding et SizedBox ont-ils exactement le même rôle ?

3. Diagnostiquer / transférer - Pourquoi Theme.of(context) dépend-il de la position du contexte ?

Ma justification : .....
.....

2. Composer et contraindre

F06 - Row, Column et composition d’une carte

Repère de cours

Row place les enfants horizontalement ; Column les place verticalement. Leur axe principal suit cette direction, et l'axe transversal lui est perpendiculaire. Une carte peut donc contenir une Row avec une icône et une Column de textes.

mainAxisAlignment agit sur l'axe principal et crossAxisAlignment sur l'axe transversal. mainAxisSize.min demande au Flex de limiter sa taille principale à ce dont ses enfants ont besoin, dans ses contraintes. Les noms se lisent par rapport au widget concerné, pas par rapport à toute la page.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
Row(
  children: [
    const CircleAvatar(child: Icon(Icons.handyman)),
    const SizedBox(width: 12),
    Expanded(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: const [Text('Gérard'), Text('Plomberie')],
      ),
    ),
  ],
)
```

Manipulation élève

Ajouter Artisan et les données du jalon 03. Dans notre catalogue, écrire _carte pour afficher le premier artisan : Row, CircleAvatar, Expanded et Column. Garder l’en-tête déjà réalisé.

Continuité : jalon 03_presentation ; guide parcours/03_presentation.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : L’en-tête et le message sont toujours là. La première carte montre Gérard, Plomberie et son tarif fictif. Un nom long reste dans la largeur disponible.

Quiz - trois questions de difficulté croissante

1. Repérer - Quel widget aligne horizontalement ?

2. Prédire - Quel est l’axe principal de cette Column ?

3. Diagnostiquer / transférer - Pourquoi un crossAxisAlignment.start dans Row et dans Column n’agit-il pas dans la même direction ?

Ma justification : .....
.....

F07 - Contraintes, Expanded et débordements

Repère de cours

Le parent transmet des contraintes ; l'enfant choisit une taille compatible ; le parent le positionne. Une largeur demandée ne gagne donc pas toujours contre les contraintes du parent. Expanded utilise l'espace restant sur l'axe principal d'un Flex, comme Row ou Column.

Dans une Row, Expanded donne une largeur disponible au texte. Dans une Column de page, il donne à la liste une hauteur bornée. Un ListView vertical placé sans hauteur bornée dans une Column provoque souvent une erreur. Ajouter shrinkWrap partout ne remplace pas la compréhension des contraintes.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
// Structure de la page :
Column(
  children: [
    const Text('En-tête'),
    Expanded(child: ListView.builder(...)),
  ],
)
```

Manipulation élève

Sur notre carte, tester un nom long et réduire la fenêtre. Expliquer Expanded dans la Row. Dessiner l’emplacement d’un second Expanded qui recevra la liste au jalon suivant.

Continuité : jalon 03_presentation ; guide parcours/03_presentation.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : L’en-tête et le message sont toujours là. La première carte montre Gérard, Plomberie et son tarif fictif. Un nom long reste dans la largeur disponible.

Quiz - trois questions de difficulté croissante

1. Repérer - Un enfant choisit-il toujours librement sa taille ?

2. Prédire - Que fournit Expanded à cette ListView ?

```
Column(children: [
  Text('Catalogue'),
  Expanded(child: ListView(...)),
])
```

3. Diagnostiquer / transférer - Pourquoi mettre Expanded dans un défilement vertical non borné n’est-il pas une solution universelle ?

Ma justification : .....
.....

3. Données, état et callbacks

F08 - Des objets Dart vers une liste défilante

Repère de cours

Les artisans sont des objets Dart constants. ListView.builder reçoit un nombre d'éléments et une fonction itemBuilder qui construit un widget pour un index demandé. Flutter construit les éléments utiles à la zone défilante et à son cache ; on ne promet pas qu'il n'existe jamais d'élément hors de l'écran.

Une donnée Artisan n'est pas un widget. Le builder lit une donnée et produit un Card ou un ListTile. Cette séparation permet de changer l'affichage sans changer le modèle métier. L'index sert à lire la liste ; une clé stable liée à l'ID peut identifier un composant lors des changements d'ordre.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
ListView.builder(
  itemCount: artisansDemo.length,
  itemBuilder: (context, index) {
    final artisan = artisansDemo[index];
    return ListTile(title: Text(artisan.nom));
  },
)
```

Manipulation élève

Garder _carte. Ajouter _liste avec ListView.builder, itemCount et l’artisan à l’index. Remplacer l’affichage d’une carte par Expanded contenant cette liste. Conserver titres, message et style de carte ; tester un quatrième artisan.

Continuité : jalon 04_catalogue ; guide parcours/04_catalogue.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : Les trois artisans initiaux sont accessibles. Le message, les titres et la présentation de chaque carte sont conservés. La liste défile sans erreur de hauteur non bornée.

Quiz - trois questions de difficulté croissante

1. Repérer - Le modèle Artisan est-il un widget ?

2. Prédire - Quelle donnée correspond à index 1 ?

3. Diagnostiquer / transférer - Pourquoi coder itemCount: 3 est-il fragile ?

Ma justification : .....
.....

F09 - StatefulWidget, State et setState

Repère de cours

StatefulWidget reste une configuration immutable. L'objet State associé conserve les données qui évoluent entre les reconstructions. La liste des IDs favoris vit dans le State, et non dans une variable locale recréée à chaque build.

setState exécute une modification synchrone puis signale que l'affichage dépendant doit être reconstruit. Il ne sauvegarde pas les données sur disque. La bonne action consiste à modifier l'état de référence puis à laisser build produire l'interface correspondante.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
setState(() {
  if (_favoris.contains(id)) {
    _favoris.remove(id);
  } else {
    _favoris.add(id);
  }
});
```

Manipulation élève

Transformer notre CatalogueScreen en StatefulWidget en déplaçant le build et ses méthodes dans le State. Ajouter _favoris, _basculerFavori avec setState, un cœur par carte et le compteur. Utiliser R après cette transformation puis tester deux clics.

Continuité : jalon 05_favoris ; guide parcours/05_favoris.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : Deux clics sur le même cœur donnent un favori puis zéro. Le compteur, les données, la carte et le message restent cohérents. Le champ _favoris reste final malgré add/remove.

Quiz - trois questions de difficulté croissante

1. Repérer - Où placer les favoris qui doivent survivre à build ?

2. Prédire - Après deux bascules du même ID, combien de favoris ?

3. Diagnostiquer / transférer - Pourquoi une modification de liste sans signalement peut-elle laisser un écran inchangé ?

Ma justification : .....
.....

F10 - Reconstruire sans perdre l’état : final et const

Repère de cours

final List<int> _favoris = [] interdit de remplacer cette référence, mais la liste peut être modifiée. Ce comportement Dart n'a pas changé. Une configuration const de widget n'interdit pas à son State associé d'évoluer.

Le State est conservé quand Flutter réutilise l'élément correspondant. Recréer une liste locale dans build n'est pas la même chose. Un hot restart recrée l'application et perd les favoris de cet exercice ; aucune persistance n'est mise en place aujourd'hui.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
class _CatalogueScreenState extends State<CatalogueScreen> {
  final List<int> _favoris = [];
  // add/remove changent cette liste ; setState demande de reconstruire.
}
```

Manipulation élève

Sur notre catalogue fonctionnel, sélectionner un favori puis recharger avec r après une modification de texte. Comparer avec R. Expliquer pourquoi la liste _favoris reste final tout en acceptant add et remove.

Continuité : jalon 05_favoris ; guide parcours/05_favoris.md. Le travail reste dans lib et le point d'entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : Deux clics sur le même cœur donnent un favori puis zéro. Le compteur, les données, la carte et le message restent cohérents. Le champ _favoris reste final malgré add/remove.

Quiz - trois questions de difficulté croissante

1. Repérer - Peut-on faire add sur une List ordinaire derrière final ?

2. Prédire - const CatalogueScreen() empêche-t-il le cœur de changer ?

3. Diagnostiquer / transférer - Quel défaut introduirait une nouvelle liste dans build ?

Ma justification : .....
.....

F11 - Extraire CarteArtisan et transmettre des callbacks

Repère de cours

Un composant reçoit ses données et signale une intention avec une fonction. CarteArtisan reçoit artisan, favori, onFavori et onOuvrir. Le parent possède la liste des favoris ; l'enfant reçoit seulement le booléen à afficher. Cela évite deux sources de vérité.

VoidCallback est un alias de void Function(). onPressed: action transmet une fonction ; onPressed: action() exécute l'action immédiatement et fournit son résultat, ce qui ne correspond généralement pas au contrat. Pour transmettre un argument plus tard, écrire () => action(id). Une ValueKey basée sur un ID stable aide à identifier un composant.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
CarteArtisan(
  artisan: artisan,
  favori: _favoris.contains(artisan.id),
  onFavori: () => _basculerFavori(artisan.id),
  onOuvrir: () => _ouvrir(artisan),
)
```

Manipulation élève

Déplacer le rendu de la carte dans widgets/carte_artisan.dart. Garder _favoris et _basculerFavori inchangés dans le catalogue. Transmettre artisan, favori et les callbacks. Le bouton Voir la fiche affiche pour le moment une SnackBar de sélection.

Continuité : jalon 06_composants ; guide parcours/06_composants.md. Le travail reste dans lib et le point d'entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : Les favoris fonctionnent toujours, sans réécrire leur logique. Deux cartes partagent un composant mais reçoivent des artisans différents. Voir la fiche affiche le nom du bon artisan dans une SnackBar.

Quiz - trois questions de difficulté croissante

1. Repérer - Qui possède la liste des favoris ?

2. Prédire - Quand basculer(id) sera-t-elle appelée ici ?

```
onPressed: () => basculer(id)
```

3. Diagnostiquer / transférer - Pourquoi onPressed: basculer(id) est-il incorrect dans ce cas ?

Ma justification : .....
.....

4. Naviguer, saisir et attendre

F12 - Naviguer vers une fiche et récupérer un résultat

Repère de cours

Navigator gère une pile de routes. push ouvre une route et renvoie un Future qui sera complété au retour ; pop ferme la route courante et peut renvoyer une valeur. MaterialPageRoute construit la page à afficher.

L'artisan est transmis par le constructeur, sans variable globale. push<String> précise le type du résultat, qui peut être absent si l'utilisateur revient sans choisir. Au retour, vérifier le contexte de vie avant de montrer un message. Le catalogue garde son State tant que sa route reste conservée.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
final message = await Navigator.of(context).push<String>(
  MaterialPageRoute(
    builder: (context) => DetailScreen(artisan: artisan),
  ),
);
if (!mounted || message == null) return;
ScaffoldMessenger.of(context)
  ..clearSnackBars()
  ..showSnackBar(SnackBar(content: Text(message)));
// Dans la fiche :
Navigator.of(context).pop('Artisan choisi');
```

Manipulation élève

Ajouter DetailScreen à notre application. Modifier seulement _ouvrir pour faire push<String>, attendre le résultat, contrôler mounted et null puis afficher le message. Dans la fiche, Choisir renvoie le nom de l’artisan. Vérifier que les favoris sont conservés au retour.

Continuité : jalon 07_navigation ; guide parcours/07_navigation.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : Chaque bouton ouvre la fiche du bon artisan. Choisir renvoie le message ; revenir en arrière peut renvoyer null. Un favori choisi avant l’ouverture est encore sélectionné au retour.

Quiz - trois questions de difficulté croissante

1. Repérer - push et pop font-ils la même chose ?

2. Prédire - Que reçoit l’appelant si la fiche revient avec pop("Choisi") ?

3. Diagnostiquer / transférer - Pourquoi faut-il traiter un résultat null ?

Ma justification : .....
.....

F13 - Form et TextFormField : valider avant d’agir

Repère de cours

Form regroupe des champs et une GlobalKey<FormState> permet de demander leur validation. TextFormField accepte un validator : il renvoie un message si la valeur n'est pas acceptable, et null si elle est valide. Une chaîne vide n'est pas null.

Le motif doit contenir au moins cinq caractères utiles, après trim. validate déclenche les validateurs et renvoie un booléen. Le service conserve aussi sa propre règle métier ; un formulaire améliore le retour utilisateur mais ne remplace pas toute validation hors de l'interface.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
validator: (valeur) {
  if (valeur == null || valeur.trim().length < 5) {
    return 'Indique au moins 5 caractères.';
  }
  return null;
}
```

Manipulation élève

Dans la même fiche, ajouter un Form, un TextFormField et son validator. Le bouton Préparer la demande valide avant de renvoyer un message. Garder les informations de l’artisan et la sélection simple. Tester vide, espaces, abc et Fuite cuisine.

Continuité : jalon 08_formulaire ; guide parcours/08_formulaire.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/detail_screen.dart

Preuve attendue : Les informations et la sélection simple de la fiche sont conservées. Un motif court ne ferme pas la fiche ; un motif valide prépare la demande. Le catalogue et ses favoris sont encore présents après le retour.

Quiz - trois questions de difficulté croissante

1. Repérer - Que renvoie un validator quand le champ est valide ?

2. Prédire - Trois espaces satisfont-ils la règle après trim ?

3. Diagnostiquer / transférer - Pourquoi un contrôle du bouton ne suffit-il pas comme règle métier unique ?

Ma justification : .....
.....

F14 - Contrôleur et cycle de vie : créer une fois, libérer

Repère de cours

TextEditingController donne accès à la saisie et suit sa valeur. Le contrôleur de cette page est créé dans le State, pas dans build. dispose libère sa ressource quand le State est retiré définitivement.

Un simple onChanged peut suffire pour certains champs, comme la recherche. Un contrôleur est utile quand l'écran doit relire ou modifier explicitement le texte. Ne pas multiplier les propriétaires d'un même contrôleur.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
final _motif = TextEditingController();

@override
void dispose() {
  _motif.dispose();
  super.dispose();
}
```

Manipulation élève

Dans le State de notre fiche, repérer la création du contrôleur, sa connexion au champ, sa lecture et sa libération dans dispose. Ouvrir puis fermer la fiche. Expliquer pourquoi on ne crée pas le contrôleur dans build.

Continuité : jalon 08_formulaire ; guide parcours/08_formulaire.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/detail_screen.dart

Preuve attendue : Les informations et la sélection simple de la fiche sont conservées. Un motif court ne ferme pas la fiche ; un motif valide prépare la demande. Le catalogue et ses favoris sont encore présents après le retour.

Quiz - trois questions de difficulté croissante

1. Repérer - Où créer le contrôleur utilisé par ce formulaire ?

2. Prédire - Quelle propriété donne le contenu saisi ?

3. Diagnostiquer / transférer - Pourquoi ne pas créer un nouveau contrôleur à chaque build ?

Ma justification : .....
.....

F15 - Afficher attente, succès et erreur avec Future

Repère de cours

Le Future de lundi devient une expérience visible : un bouton lance une opération, un indicateur montre l'attente, puis le succès ou l'erreur est présenté. Le bouton est désactivé pendant l'envoi pour éviter une répétition involontaire.

Le callback de setState reste synchrone. L'opération attendue se déroule dans une méthode async, en dehors de ce callback. try/catch/finally garde des chemins distincts. La simulation du jour n'envoie rien à un artisan et ne stocke aucune demande. Une panne conserve le motif et rend une nouvelle tentative possible.

On enrichit le formulaire déjà validé : le contrôleur, le validator et dispose ne sont pas réécrits. Le bouton de préparation devient un envoi simulé ; la sélection simple reste une action distincte.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
setState(() => _envoi = true);
try {
  final resultat = await simulerEnvoi(...);
  // Présenter le résultat après les vérifications de cycle de vie.
} finally {
  if (mounted) setState(() => _envoi = false);
}
```

Manipulation élève

Conserver le contrôleur et le validator. Ajouter le service simulé et faire évoluer _envoyer avec _envoi, _erreur, await, catch et finally. Observer le chargement et le bouton désactivé. Activer la panne dans HelpMeApp, utiliser R, puis revenir au succès avec false et R.

Continuité : jalon 09_asynchrone ; guide parcours/09_asynchrone.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/detail_screen.dart

Preuve attendue : La validation et le contenu de la fiche sont conservés. Le chargement est visible et un double clic ne lance pas un second envoi. La panne garde le motif et réactive le bouton. Revenir pendant l’attente laisse le catalogue visible à la fin du délai.

Quiz - trois questions de difficulté croissante

1. Repérer - Que représente Future<String> ?

2. Prédire - Pourquoi mettre onPressed à null pendant l’envoi ?

3. Diagnostiquer / transférer - Pourquoi ne pas écrire setState(() async { await service(); }) ?

Ma justification : .....
.....

F16 - mounted et retour arrière pendant une attente

Repère de cours

Après await, l'écran peut avoir été fermé. mounted indique si son State est encore attaché à l'arbre. Il faut le vérifier avant d'utiliser son contexte ou de demander un changement d'état après cette attente.

Pendant l'animation de retour, le State peut rester monté alors que sa route n'est déjà plus courante. La correction vérifie aussi ModalRoute.of(context)?.isCurrent avant de faire pop sur un succès. Cette garde évite de fermer par erreur un autre écran ; elle est fournie, sans demander de mémoriser son API.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
if (!mounted) return;
if (ModalRoute.of(context)?.isCurrent != true) return;
Navigator.of(context).pop(resultat);
```

Manipulation élève

Avec notre catalogue et sa fiche, commencer un envoi puis revenir aussitôt. Attendre sa fin. Garder les protections mounted et route courante fournies dans le code. Vérifier que le catalogue reste ouvert et que son favori est conservé.

Continuité : jalon 09_asynchrone ; guide parcours/09_asynchrone.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/detail_screen.dart

Preuve attendue : La validation et le contenu de la fiche sont conservés. Le chargement est visible et un double clic ne lance pas un second envoi. La panne garde le motif et réactive le bouton. Revenir pendant l’attente laisse le catalogue visible à la fin du délai.

Quiz - trois questions de difficulté croissante

1. Repérer - Que signifie mounted pour ce State ?

2. Prédire - Peut-on supposer que l’écran existe encore après await ?

3. Diagnostiquer / transférer - Pourquoi mounted seul ne suffit-il pas toujours avant un pop ?

Ma justification : .....
.....

5. Adapter, vérifier et intégrer

F17 - Adapter le layout à la largeur disponible

Repère de cours

LayoutBuilder reçoit les contraintes disponibles à l'endroit où il se trouve. La correction choisit une colonne sous 700 pixels logiques et deux colonnes à partir de cette largeur. Le critère dépend de l'espace disponible, pas du nom du téléphone.

Une seule source de données et de favoris alimente les deux présentations. Les cartes gardent leur hauteur naturelle pour laisser passer les textes longs ou agrandis. Un layout responsive n'est pas une preuve de fonctionnement natif sur un appareil réel.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
LayoutBuilder(
  builder: (context, contraintes) {
    final large = contraintes.maxWidth >= 700;
    return large ? listeDeuxColonnes : listeUneColonne;
  },
)
```

Manipulation élève

Modifier seulement _liste dans notre catalogue pour utiliser LayoutBuilder et une ou deux colonnes. Garder les mêmes favoris, la même carte et la fiche. Modifier la largeur sans relancer ; vérifier qu’un favori reste sélectionné.

Continuité : jalon 10_responsive ; guide parcours/10_responsive.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : Les favoris et la fiche fonctionnent à petite et grande largeur. Changer la largeur ne réinitialise pas les favoris. Les textes agrandis et les erreurs restent accessibles.

Quiz - trois questions de difficulté croissante

1. Repérer - Quelle information LayoutBuilder fournit-il ici ?

2. Prédire - Avec le seuil 700, combien de colonnes à 600 puis 900 ?

3. Diagnostiquer / transférer - Pourquoi éviter deux listes de favoris pour les deux layouts ?

Ma justification : .....
.....

F18 - Accessibilité : nommer les actions et agrandir le texte

Repère de cours

Une icône seule ne suffit pas à expliquer une action. Les boutons favoris possèdent un tooltip qui nomme la personne et le sens de l'action. Le formulaire garde un label explicite et une erreur textuelle ; le sens n'est pas indiqué seulement par une couleur.

Le texte doit pouvoir grandir sans disparaître dans une hauteur fixe. La fiche peut défiler avec le clavier. Semantics peut exposer une information utile aux technologies d'assistance ; liveRegion aide à annoncer une erreur qui change. Une vérification réelle au clavier ou lecteur d'écran complète les tests automatiques.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
IconButton(
  tooltip: 'Ajouter Gérard aux favoris',
  onPressed: action,
  icon: const Icon(Icons.favorite_border),
)
```

Manipulation élève

Dans notre application, lire le tooltip d’un favori, agrandir le texte et vérifier l’accès aux cartes. Sur la fiche, contrôler les erreurs de validation et de service ainsi que le défilement. Garder tous les comportements précédents.

Continuité : jalon 10_responsive ; guide parcours/10_responsive.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : Les favoris et la fiche fonctionnent à petite et grande largeur. Changer la largeur ne réinitialise pas les favoris. Les textes agrandis et les erreurs restent accessibles.

Quiz - trois questions de difficulté croissante

1. Repérer - Une couleur rouge suffit-elle à expliquer une erreur ?

2. Prédire - Quel libellé est plus utile : Favori ou Ajouter Gérard aux favoris ?

3. Diagnostiquer / transférer - Pourquoi une hauteur fixe peut-elle devenir problématique avec du texte agrandi ?

Ma justification : .....
.....

F19 - Lire une erreur et vérifier un comportement

Repère de cours

L'analyseur détecte des problèmes de code avant l'exécution. Une erreur de layout ou de comportement demande aussi une observation de l'interface. Lire la première erreur utile et son fichier permet d'éviter une suite de changements sans hypothèse.

Un widget test construit un écran, agit et vérifie un résultat. pump affiche une évolution ; pumpAndSettle attend que les animations se stabilisent. Pendant un indicateur qui tourne en continu, attendre sans limite la stabilisation n'est pas une bonne stratégie : on fait avancer le délai contrôlé puis on attend les animations finales.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
await tester.tap(find.byKey(const ValueKey('favori_1')));
await tester.pump();
expect(find.text('1 favori(s)'), findsOneWidget);
```

Manipulation élève

Lire le test donné dans le PDF, décrire son action et sa preuve. Reproduire le geste sur notre application. Le test du kit de départ se lance avec flutter test ; les vérifications complètes des corrections servent à la démonstration.

Continuité : jalon 10_responsive ; guide parcours/10_responsive.md. Le travail reste dans lib et le point d'entrée reste lib/main.dart.

Lire ici l'extrait et réaliser le geste dans notre app. Le kit possède un test de démarrage ; les vérifications de tous les jalons sont dans le kit de correction.

Preuve attendue : Les favoris et la fiche fonctionnent à petite et grande largeur. Changer la largeur ne réinitialise pas les favoris. Les textes agrandis et les erreurs restent accessibles.

Quiz - trois questions de difficulté croissante

1. Repérer - flutter analyze prouve-t-il que tous les boutons fonctionnent ?

2. Prédire - Que vérifie cette assertion après un clic de favori ?

```
expect(find.text('1 favori(s)'), findsOneWidget);
```

3. Diagnostiquer / transférer - Pourquoi pumpAndSettle peut-il attendre sans fin pendant un chargement animé ?

Ma justification : .....
.....

F20 - Atelier autonome : relier recherche et favoris

Repère de cours

La recherche s'ajoute à l'application déjà construite. Les favoris, les callbacks, la navigation et le formulaire restent en place. Le travail nouveau consiste à ajouter deux critères d'affichage puis un getter qui dérive la liste visible sans supprimer les données source.

La bascule d'un favori n'est pas un nouvel exercice ici : on conserve _basculerFavori. Le trajet événement → état → interface est réutilisé pour la saisie et le filtre.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
final texte = _recherche.trim().toLowerCase();
return artisansDemo.where((artisan) {
  final correspond = artisan.nom.toLowerCase().contains(texte) ||
      artisan.metier.toLowerCase().contains(texte);
  final favoriAccepte = !_seulementFavoris || _favoris.contains(artisan.id);
  return correspond && favoriAccepte;
}).toList();
```

Manipulation élève

Ajouter _recherche, _seulementFavoris, TextField, FilterChip et le getter _visibles. Conserver _basculerFavori et _ouvrir : ils sont déjà terminés. Combiner recherche et favoris sans modifier artisansDemo, afficher un résultat vide puis effacer les filtres.

Continuité : jalon 11_recherche ; guide parcours/11_recherche.md. Le travail reste dans lib et le point d'entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : Recherche seule, favoris seuls et combinaison donnent la bonne personne. Effacer la recherche retrouve toutes les données sans perte. Le compteur, la navigation, la validation, l'attente, l'erreur et le responsive fonctionnent encore.

Quiz - trois questions de difficulté croissante

1. Repérer - Le filtre doit-il supprimer des données dans artisansDemo ?

2. Prédire - Si Sophie est favorite et que la recherche vaut sophie, que doit afficher la combinaison ?

3. Diagnostiquer / transférer - Pourquoi garder un état séparé visibleArtisans peut-il compliquer cette app ?

Ma justification : .....
.....

F21 - Recette finale et transmission vers mercredi

Repère de cours

Une preuve complète suit un parcours : trouver un artisan, le mettre en favori, ouvrir sa fiche, refuser un motif court, attendre un envoi valide et observer le retour. Une autre preuve couvre la panne sans perte de saisie.

Si un appareil réel est déjà préparé, exécuter le parcours au tactile et en orientation différente. Sinon, tester l'adaptation sur Chrome et noter le contrôle natif restant à reprendre. Aujourd'hui, favoris et demandes ne sont pas persistés. Mercredi pourra reprendre état partagé, données et stockage à partir de ces limites identifiées.

Extrait à lire et à prédire

Extrait pédagogique : dépend du contexte indiqué. Le fichier du projet contient les imports et la structure exécutables. Les points de suspension ne se recopient pas comme du code complet.

```
# Avec appareil et plateformes natives déjà préparés :
flutter devices
flutter run -d IDENTIFIANT_APPAREIL
# Sur Chrome :
flutter run -d chrome
```

Manipulation élève

Depuis le même main, réaliser la recette complète en binôme : favoris, fiche, sélection, validation, attente, réussite et panne, puis largeurs différentes. Expliquer une contrainte, un callback et une attente. Si un appareil est prêt, lancer la même app dessus.

Continuité : jalon 11_recherche ; guide parcours/11_recherche.md. Le travail reste dans lib et le point d’entrée reste lib/main.dart.

Repère fichier : lib/screens/catalogue_screen.dart

Preuve attendue : Recherche seule, favoris seuls et combinaison donnent la bonne personne. Effacer la recherche retrouve toutes les données sans perte. Le compteur, la navigation, la validation, l’attente, l’erreur et le responsive fonctionnent encore.

Quiz - trois questions de difficulté croissante

1. Repérer - Les favoris de cet atelier survivent-ils automatiquement à un redémarrage ?

2. Prédire - Pourquoi une panne ne doit-elle pas effacer le motif ?

3. Diagnostiquer / transférer - Quelle preuve manque si l’app a seulement tourné dans Chrome sur ordinateur ?

Ma justification : .....
.....

Récapitulatif du cours suivi

Notion	Repère
Dart / Flutter	Le langage porte les règles ; le framework décrit et affiche l’interface.
Widget / build	Configuration en arbre ; build renvoie un Widget et peut être rappelé.
child / children	Un enfant / une liste de widgets.
Stateless / Stateful	Configuration seule / configuration associée à un State mutable.
Contraintes	Le parent contraint ; l’enfant choisit sa taille ; le parent positionne.
Expanded	Partage l’espace restant sur l’axe principal d’un Flex adapté.
Builder	Une fonction construit le widget d’un index demandé.
setState	Modification synchrone et demande de reconstruction ; pas de persistance.
Callback	Transmettre une fonction à appeler plus tard ; l’état a un propriétaire.
Navigation	push ouvre ; pop ferme et peut renvoyer un résultat nullable.
Validation	Message String si invalide ; null si valide.
Cycle de vie	Contrôleur dans le State ; dispose ; vérifications après await.
Future	Attente, succès et erreur distincts ; pas d’async dans setState.
Accessibilité	Nommer les actions, garder les erreurs lisibles, agrandir et tester.

Lexique à garder sous la main

Terme	Sens dans le cours
Widget	Description d’une partie de l’interface
build	Construire la description attendue
State	État associé et conservé entre reconstructions
callback	Fonction appelée lors d’un événement
constraints	Contraintes de taille
padding	Espace intérieur
tooltip	Libellé d’aide d’une action
validator	Fonction qui signale une valeur invalide
mounted	Encore attaché à l’arbre
hot reload / hot restart	Recharger le code / redémarrer l’app Dart

Checklist de recette et reprise personnelle

Preuve	À reprendre / Avec indice / Autonome
Un écran et un arbre expliqués	.....
Liste défilante sans débordement	.....
Favori et filtre cohérents	.....
La bonne fiche reçoit les bonnes données	.....
Motif invalide refusé et erreur lisible	.....
Attente visible, panne sans perte de saisie	.....
Retour pendant attente sans écran perdu	.....
Largeur et texte agrandi vérifiés	.....
Vérification sur appareil réel identifiée	.....

Ma reprise prioritaire pour mercredi : .....
.....

Corrigés des quiz - après sa tentative

F01 - De Dart à Flutter : lancer un premier écran

1. Non : Dart est le langage, Flutter le framework d’interface.
Pourquoi : Les règles métier en Dart peuvent exister sans écran Flutter.

2. Text dans l’interface ; debugPrint dans la console.
Pourquoi : Construire un Text ne suffit pas : il doit aussi être inséré dans l’arbre affiché. Un log aide à observer l’exécution.

3. L’application utilise les bibliothèques et le moteur Flutter.
Pourquoi : Le CLI de lundi était du Dart seul. Lancer ici un point d’entrée Flutter depuis son projet.

F02 - Enregistrer, hot reload et hot restart

1. r.
Pourquoi : R, en majuscule, demande un hot restart.

2. Oui, généralement.
Pourquoi : Le rechargement conserve le State existant ; ce n’est pas un stockage durable.

3. Hot restart ou relance complète selon le changement.
Pourquoi : Hot reload ne réexécute pas main. Il ne faut pas confondre code mis à jour et état recréé.

F03 - Widget, arbre, build et BuildContext

1. Widget.
Pourquoi : La méthode décrit une portion d’interface utilisable dans l’arbre.

2. Center.
Pourquoi : child introduit un enfant ; les parenthèses montrent l’imbrication.

3. build peut être rappelé et provoquer plusieurs envois.
Pourquoi : Une action utilisateur ou un cycle de vie approprié doit déclencher l’opération, pas la description répétée de l’interface.

F04 - StatelessWidget et paramètres nommés

1. Non.
Pourquoi : Il ne possède pas de State mutable associé ; son build peut être rappelé avec une nouvelle configuration.

2. Une liste de widgets, List<Widget>.
Pourquoi : Les crochets construisent le conteneur de plusieurs enfants.

3. Le constructeur impose required titre.
Pourquoi : La règle est vérifiée par le type et le constructeur, avant l’affichage.

F05 - MaterialApp, Scaffold, thème et premiers widgets

1. Scaffold.
Pourquoi : MaterialApp fournit l’environnement de l’application ; Scaffold structure une page.

2. Non.
Pourquoi : Padding ajoute un espace autour de son enfant ; SizedBox réserve une taille ou un espace explicite.

3. Il recherche un thème parmi ses ancêtres.
Pourquoi : Un BuildContext n’est pas une variable globale qui voit n’importe quel service dans toute l’application.

F06 - Row, Column et composition d’une carte

1. Row.
Pourquoi : Column aligne verticalement.

2. L’axe vertical.
Pourquoi : Le sens principal vient du widget Flex concerné, pas du nom de ses enfants.

3. Leurs axes transversaux sont différents.
Pourquoi : Le transversal est vertical dans Row et horizontal dans Column.

F07 - Contraintes, Expanded et débordements

1. Non.
Pourquoi : Il doit respecter les contraintes reçues de son parent.

2. La hauteur restante et bornée de la Column.
Pourquoi : La liste peut ainsi définir son viewport défilant.

3. Il peut ne pas exister de hauteur finie à partager.
Pourquoi : Expanded a besoin d’un Flex avec un espace principal cohérent ; il ne crée pas une borne magique.

F08 - Des objets Dart vers une liste défilante

1. Non.
Pourquoi : Il contient les données métier ; un widget les présente.

2. La deuxième donnée de la liste.
Pourquoi : Comme lundi, l’indexation commence à zéro.

3. Le nombre devient faux si les données changent.
Pourquoi : Utiliser artisansDemo.length garde le contrat du builder cohérent avec la liste.

F09 - StatefulWidget, State et setState

1. Dans le State de leur propriétaire.
Pourquoi : Une variable locale de build serait recréée pendant une reconstruction.

2. Zéro.
Pourquoi : La deuxième bascule retire l’ID ajouté par la première.

3. Flutter n’est pas informé qu’une reconstruction est nécessaire.
Pourquoi : Le partage de références Dart ne remplace pas le mécanisme d’état de l’interface.

F10 - Reconstruire sans perdre l’état : final et const

1. Oui.
Pourquoi : Comme lundi, final fixe la référence et ne rend pas cet objet liste immutable.

2. Non.
Pourquoi : La configuration du widget est constante ; son State peut évoluer.

3. Les données locales seraient recréées à chaque passage.
Pourquoi : L'état durable entre reconstructions doit appartenir au State approprié.

F11 - Extraire CarteArtisan et transmettre des callbacks

1. Le parent commun qui décide de l'état.
Pourquoi : La carte reçoit une valeur et un callback ; elle ne doit pas créer une seconde liste indépendante.

2. Lors de l'événement, pas pendant build.
Pourquoi : La fonction anonyme capture l'ID et sera appelée plus tard.

3. L'appel est exécuté immédiatement et ne fournit pas le callback attendu.
Pourquoi : Le paramètre attend une fonction sans argument à appeler au clic, pas le résultat void de l'appel.

F12 - Naviguer vers une fiche et récupérer un résultat

1. Non : push ouvre une route ; pop ferme la route courante.
Pourquoi : Le navigateur est ici expliqué comme une pile.

2. La valeur Choisi.
Pourquoi : Le Future de push<String> est complété avec ce résultat.

3. L'utilisateur peut revenir sans choisir.
Pourquoi : Un résultat absent n'est pas forcément une erreur ; il ne faut pas annoncer une sélection inexistante.

F13 - Form et TextFormField : valider avant d'agir

1. null.
Pourquoi : Une String représente un message d'erreur à afficher.

2. Non.
Pourquoi : trim enlève ces espaces ; il ne reste aucun caractère utile.

3. D'autres appels peuvent contourner l'interface.
Pourquoi : Le service ou modèle doit aussi protéger son contrat indépendamment du formulaire.

F14 - Contrôleur et cycle de vie : créer une fois, libérer

1. Dans le State, hors de build.
Pourquoi : Il doit rester cohérent entre deux reconstructions.

2. controller.text.
Pourquoi : La propriété du contrôleur contient la valeur actuelle du champ.

3. On recrée une ressource et on risque de perdre la continuité de saisie.
Pourquoi : build peut être rappelé souvent ; le cycle de vie doit rester explicite.

F15 - Afficher attente, succès et erreur avec Future

1. Un texte à venir ou une erreur.
Pourquoi : Ce n'est pas encore la valeur à afficher immédiatement.

2. Pour désactiver le bouton et éviter un deuxième déclenchement.
Pourquoi : L'état d'attente doit avoir un effet visible et fonctionnel.

3. Le callback de setState doit être synchrone.
Pourquoi : Mettre les petits changements d’état dans setState, puis attendre le Future dans la méthode async.

F16 - mounted et retour arrière pendant une attente

1. Il est encore attaché à l’arbre.
Pourquoi : Cela ne veut pas dire que sa route est forcément au premier plan.

2. Non.
Pourquoi : L’utilisateur peut avoir fermé l’écran pendant l’attente.

3. Une route en animation de fermeture peut encore être montée.
Pourquoi : Vérifier aussi la route courante évite un deuxième pop sur une navigation déjà quittée.

F17 - Adapter le layout à la largeur disponible

1. Les contraintes disponibles pour ce sous-arbre.
Pourquoi : Ce n’est pas obligatoirement toute la taille de l’écran.

2. Une puis deux.
Pourquoi : La condition lit une largeur logique disponible.

3. Elles pourraient diverger.
Pourquoi : La présentation peut varier tout en partageant la même source d’état.

F18 - Accessibilité : nommer les actions et agrandir le texte

1. Non.
Pourquoi : Un message textuel doit aussi permettre de comprendre et corriger le problème.

2. Ajouter Gérard aux favoris.
Pourquoi : Il donne la cible et le sens de l’action.

3. Le contenu peut dépasser ou être coupé.
Pourquoi : La mise en page doit permettre l’agrandissement, l’adaptation ou le défilement.

F19 - Lire une erreur et vérifier un comportement

1. Non.
Pourquoi : L’analyse statique ne remplace pas les tests de comportement et la recette.

2. Que le compteur visible indique un favori.
Pourquoi : Elle contrôle un résultat utilisateur observable.

3. L’animation continue à produire de nouvelles images.
Pourquoi : Faire avancer le délai de l’opération, puis attendre les animations finales.

F20 - Atelier autonome : relier recherche et favoris

1. Non.
Pourquoi : Il calcule un résultat visible et conserve les données de référence.

2. Sophie uniquement.
Pourquoi : Elle satisfait à la fois le critère de recherche et celui des favoris.

3. Il peut devenir incohérent avec la recherche et les favoris.
Pourquoi : Ici la liste visible se dérive de l’état existant ; on évite de synchroniser inutilement plusieurs sources.

F21 - Recette finale et transmission vers mercredi

1. Non.
Pourquoi : Ils vivent en mémoire ; aucun stockage persistant n’a été ajouté.

2. L’utilisateur doit pouvoir corriger ou réessayer sans tout ressaisir.
Pourquoi : L’état de la saisie et l’état d’une tentative d’envoi sont distincts.

3. La vérification sur un appareil réel et, pour le natif, son exécution native.
Pourquoi : Un viewport étroit ne prouve pas à lui seul le fonctionnement tactile ou natif.

Fichiers organisés et commandes utiles

Les chemins correspondent au dossier helpme_mardi du kit. Les fichiers de lib sont partagés par les étapes. Garder le nom de package helpme_mardi ou adapter les imports package: de façon cohérente.

lib

lib/app.dart

lib/configuration.dart

lib/main.dart

lib/screens/catalogue_screen.dart

test

test/demarrage_test.dart

Les modules à créer apparaissent dans les consignes de chaque jalon. Les blocs complets de référence sont dans parcours/ ; ils ne sont pas tous présents dans le lib initial.

```
flutter analyze
flutter test
flutter run -d chrome
```

Repères officiels et suite du parcours

Les API et concepts utilisés sont expliqués dans les sources officielles ci-dessous. Ce cours pose les premiers écrans et interactions. Les données distantes, un état partagé plus large, la persistance et la publication se traitent après cette base.

Premiers widgets

Hot reload et hot restart

Contraintes et layout

setState

mounted

Navigation push et pop

Formulaires et validation

Widget tests

Accessibilité

Layouts adaptatifs